

From the output, it can be seen that the file format of the executable *study* is ELF. What GCC has done is, it has passed *study.c* through the following stages of compilation to convert the *study.c* file to a final executable:

- Pre-processing
- Compiling (assembly file creation)
- Assembler (object file creation)
- Linking (links all object files to create an executable)

So, let's explore each stage to understand how each process works in compilation.

Pre-processing

This is the first stage of compilation in which the source code is processed.

The following high-level tasks are done at the pre-processing stage:

- Removal of comments from source code
- Expanding *#include* files
- Macro expansion
- Conditional compilation

Now let's try to pre-process our *study.c* file and look at its output. The following is the command by which we can restrict the compiler to just pre-process the *study.c* file and generate the pre-processed file instead of the executable:

```
gcc -E study.c -o study.i
```

Here, *study.i* is the pre-processed file. Open the *study.i* file, and you should see the following contents:

```
[lines omitted for brevity]
extern void funlockfile (FILE *__stream)
__attribute__ ((__nothrow__ , __leaf__));
# 868 "/usr/include/stdio.h" 3 4
# 2 "study.c" 2
# 6 "study.c"
int main(void)
{
    int a,b,c;
    a = 1;
    b = 2;
    c = a + b;
```

```
printf("Output: %d\r\n", c);
return 0
}
```

From the *study.i* file, it can be seen that the statement *#include <stdio.h>* has been replaced by the contents of the *stdio.h* file. Macros *NUMBER_1* and *NUMBER_2* are not present in the output. Besides, wherever these macros were used, their values are replaced by macro values. So *a*

= *NUMBER_1* is replaced by *a = 1*.

In this compilation stage, any line that starts with *#* is interpreted as a pre-processing command. These commands have their own language format (which is not discussed in detail here since online sources describe it in detail).

 **Note:** Try to write different macros and conditional compilation commands in the code, then run the provided pre-processing command and see the output in order to understand pre-processing in detail.

Compilation

What this stage is called is confusing, as it converts pre-processed code into assembly code. In this stage, all programming symbols are converted to assembly instructions, which are native to the target processor architecture. This is also the stage where inline assembly code in high-level programming languages is processed.

Now let's compile our *study.c* file and produce the equivalent assembly instruction file. The following command restricts the compiler to convert the *study.c* file to an assembly file.

```
gcc -S study.c -fverbose-asm -o study.s
```

After running the above command, open the *study.s* file, which contains the assembly equivalent of the program. This file will also show assembly code generated for each line of C code that

we have written. Using this, you can understand how assembly instructions and C programming syntax are related.

 **Note:** Modify the above program and run the above command to see how assembly instructions are generated for different C syntaxes.

Assembling

This stage is used to convert assembly instructions to machine code or object code. This object code is the actual machine instructions that run on the target processor. This is the first stage in which the ELF file is generated. The output file is known as an *object* file which has an extension *.o* and has the file format ELF. This file cannot be directly executed, but needs to be linked using Linker, which is the final stage of compilation.

To convert our *study.c* file to an object file, we will use the following command:

```
gcc -c study.c -o study.o
```

study.o cannot be directly opened as it is a binary file. To open this file, we can use binary utilities.

ELF files and binary utilities

To understand the final stage of compilation, which is linking, and to visualise how ELF files are structured, we need to understand object files, particularly ELF files. We were able to generate object files in the previous stage, but we were not able to open them. To open this file, we need to understand the structure of an object file.

ELF object files contain five kinds of information:

- ELF header
- Section header and different sections
- Program header and different program sections
- Object code
- Debugging information